CS-2001
Data Structures
Fall 2023

Graph Traversals

Rizwan Ul Haq

Assistant Professor

FAST-NU

rizwan.haq@nu.edu.pk

Graph Searching

- Given: a graph G = (V, E), directed or undirected
- Goal: methodically explore every vertex and every edge
- Two main approaches:
  - Breadth-First Search
  - Depth-First Search

Breadth First Search

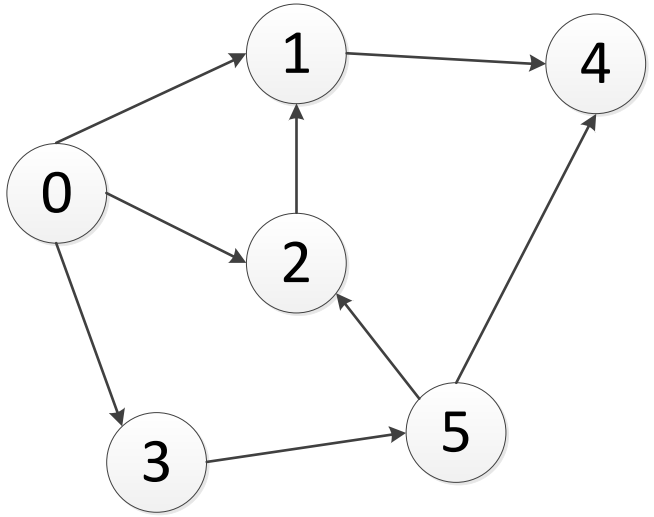
- The algorithm discovers all vertices at distance k from s before discovering any vertices at distance k+1

Breadth First Search

- The algorithm uses a queue data structure to store intermediate results as it traverses the graph, as follows:
  1. Enqueue the source node
  2. Dequeue a node and examine it
     - If the element required is found in this node, quit the search and return a result
     - Otherwise enqueue any successors (the direct child nodes) that have not yet been discovered
  3. If the queue is empty, every node on the graph has been examined – quit the search and return "not found".
  4. If the queue is not empty, repeat from Step 2

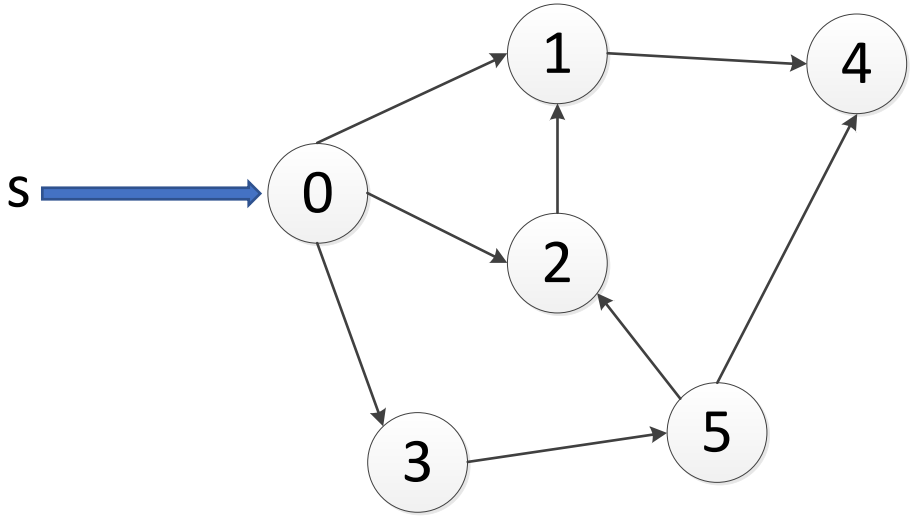
Breadth First Algorithm

```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Breadth First Algorithm

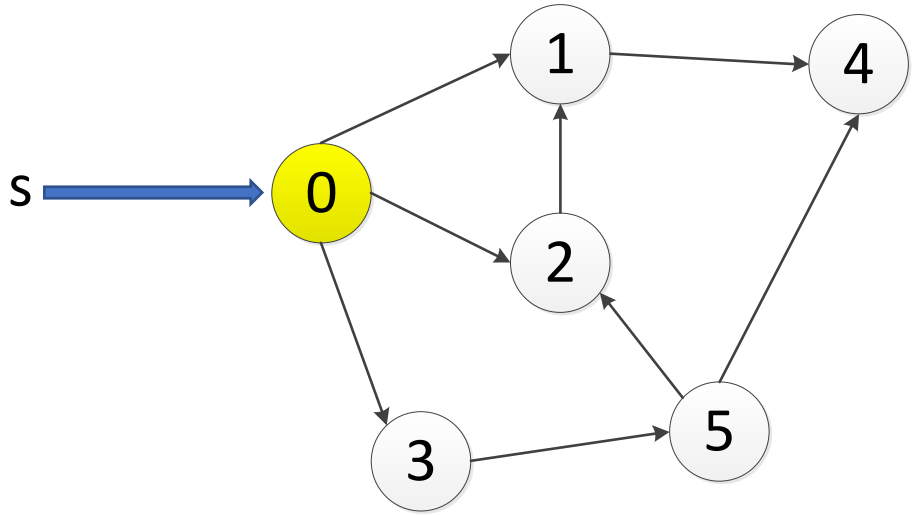
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = ∅

Breadth First Algorithm

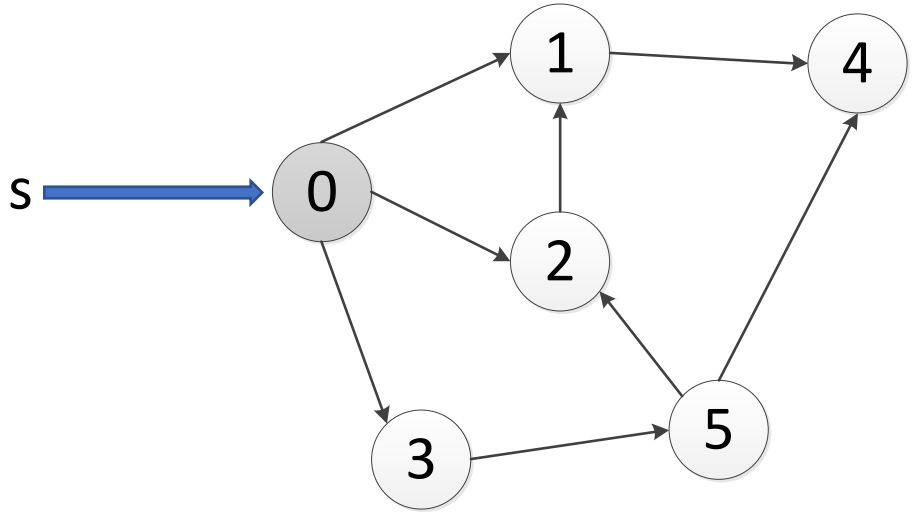
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = ∅

Breadth First Algorithm

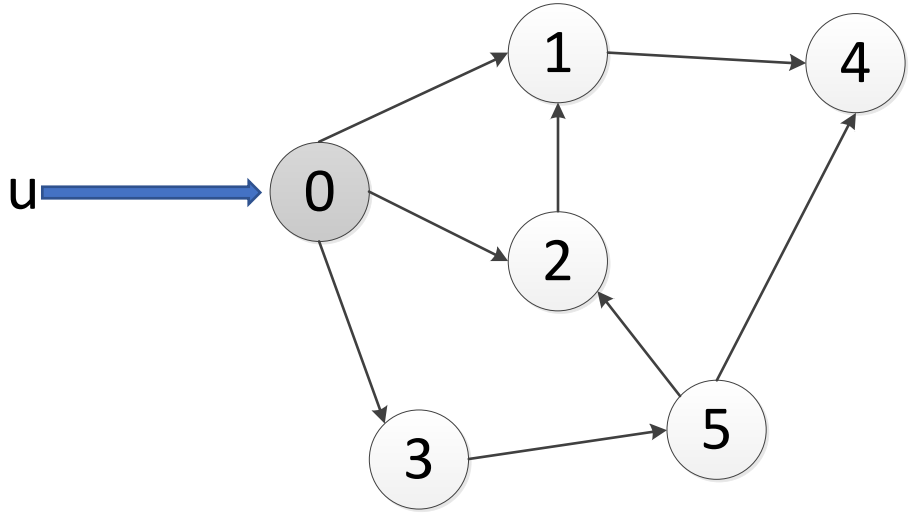
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = 0

Breadth First Algorithm

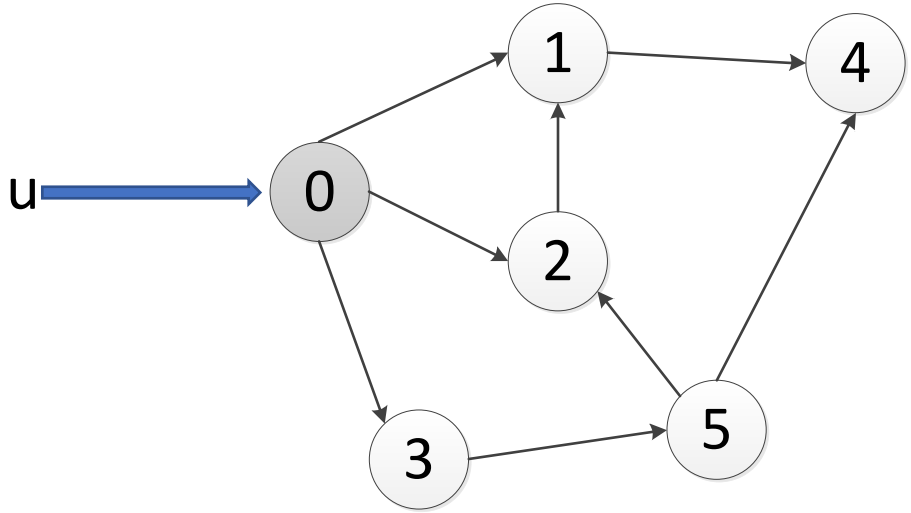
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = 0

Breadth First Algorithm

```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```

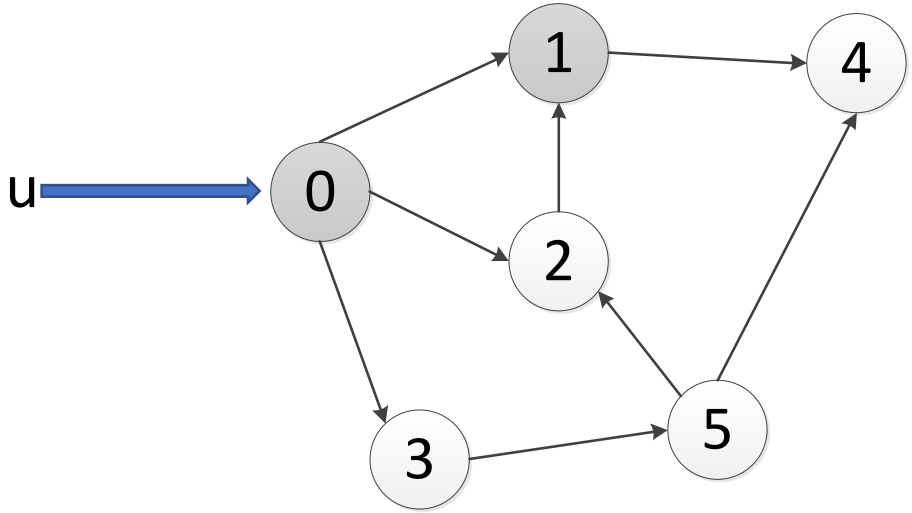


Q =

0	1

Breadth First Algorithm

```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```

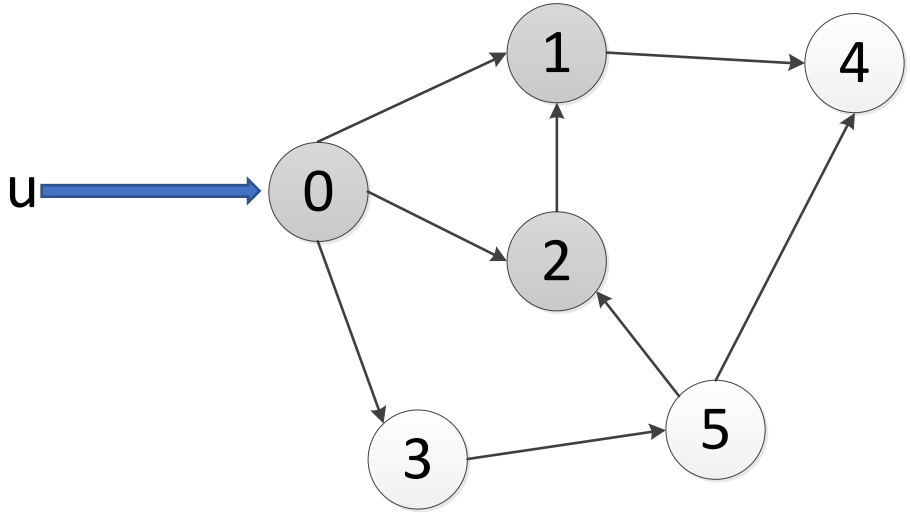


Q =

0	1

Breadth First Algorithm

```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```

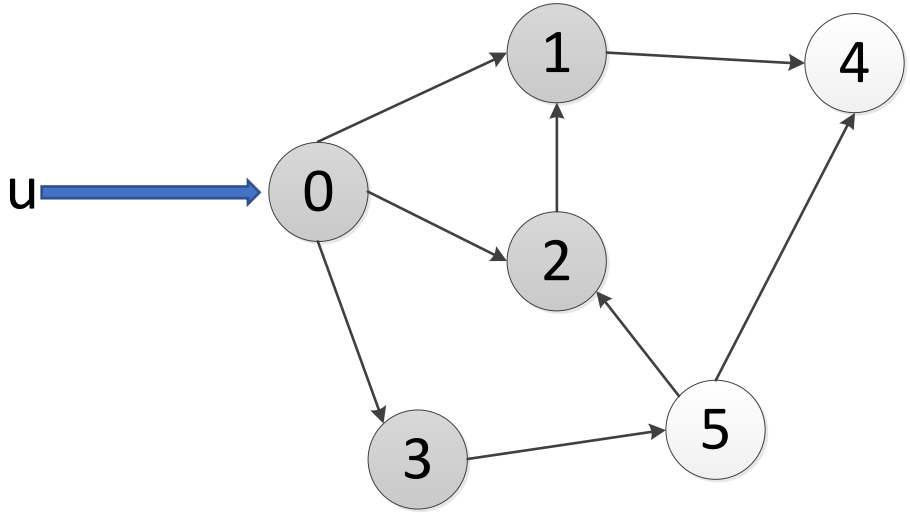


Q =

0	1	2

Breadth First Algorithm

```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```

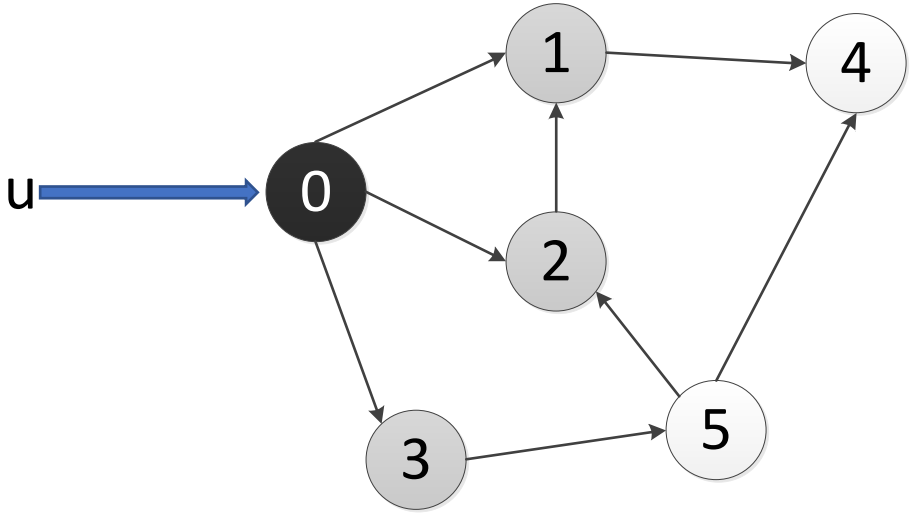


Q =

0	1	2	3

Breadth First Algorithm

```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```

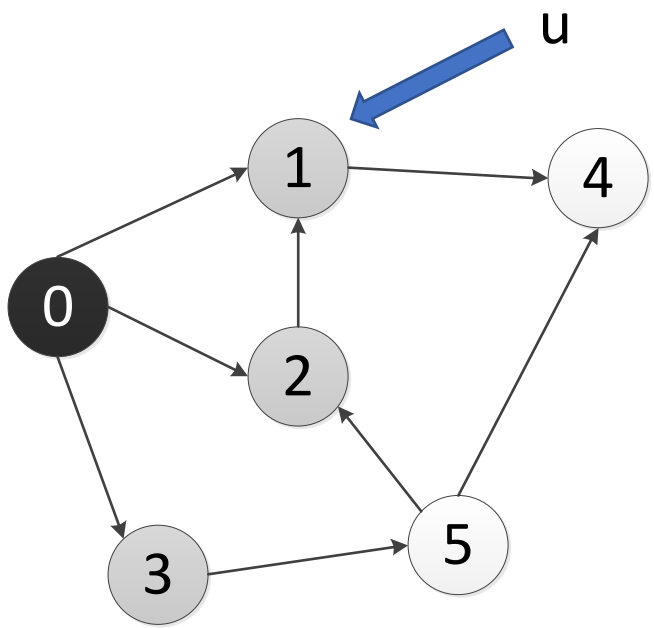


Q =

1	2	3

Breadth First Algorithm

```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```

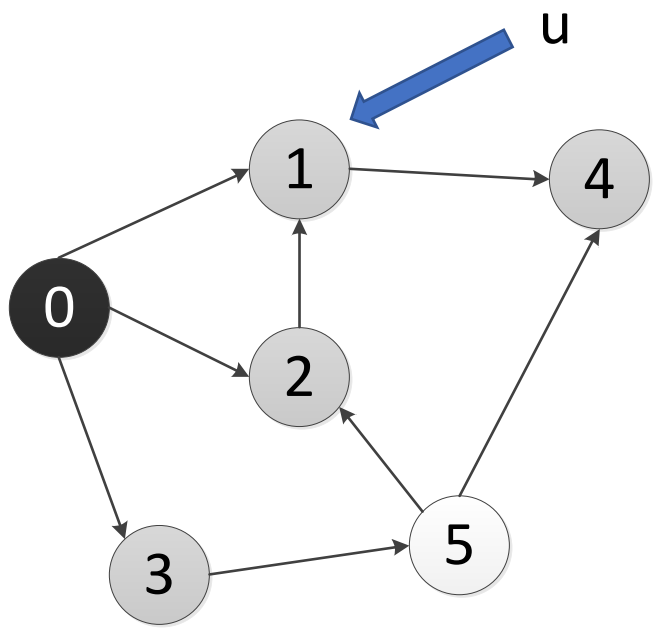


Q =

1	2	3

Breadth First Algorithm

```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```

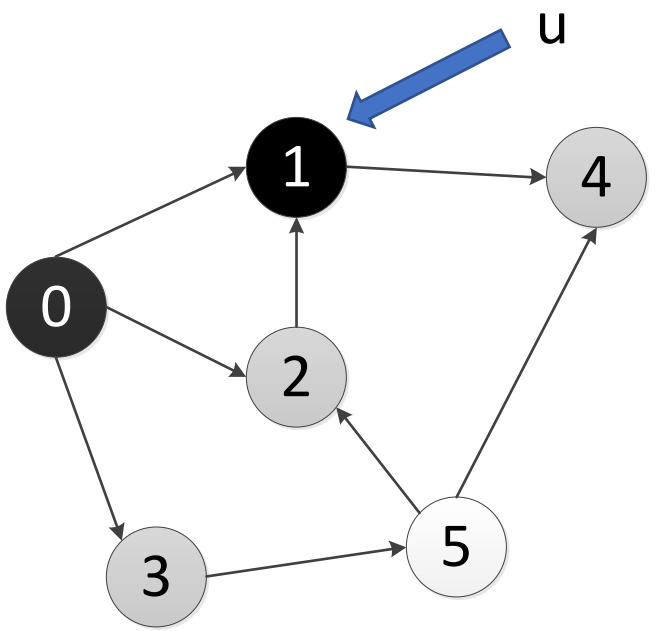


Q =

1	2	3	4

Breadth First Algorithm

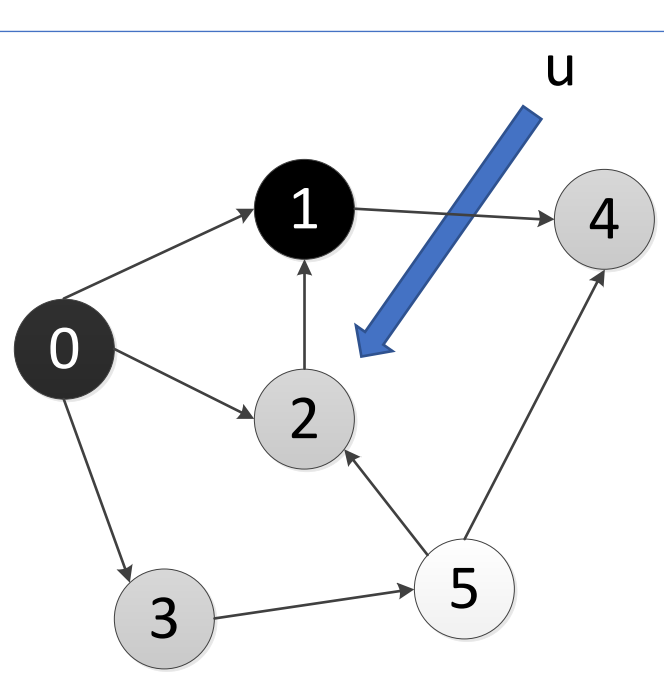
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = | 2 | 3 | 4 |

Breadth First Algorithm

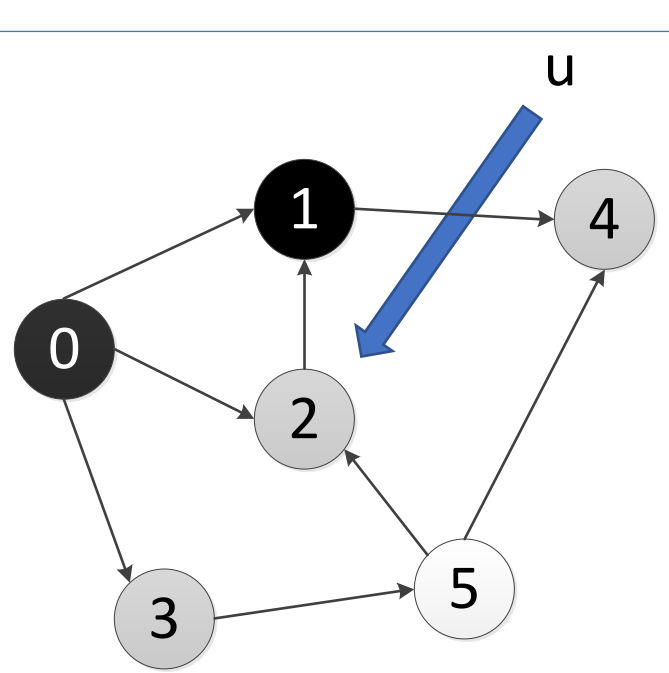
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = | 2 | 3 | 4 |

Breadth First Algorithm

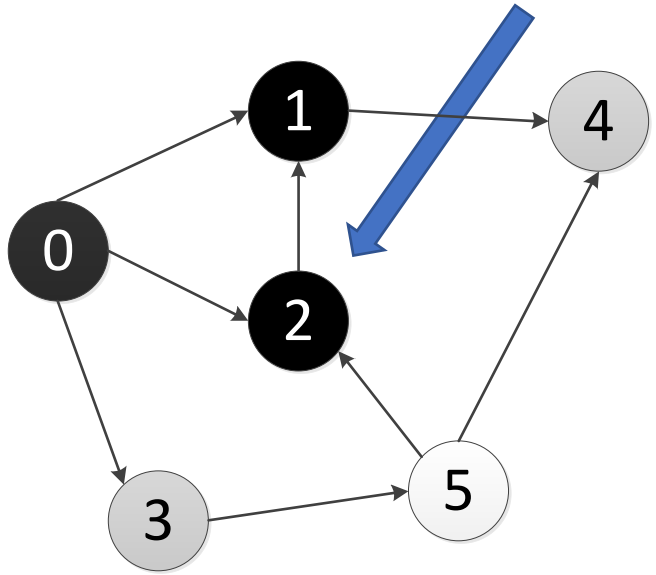
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = | 2 | 3 | 4 |

Breadth First Algorithm

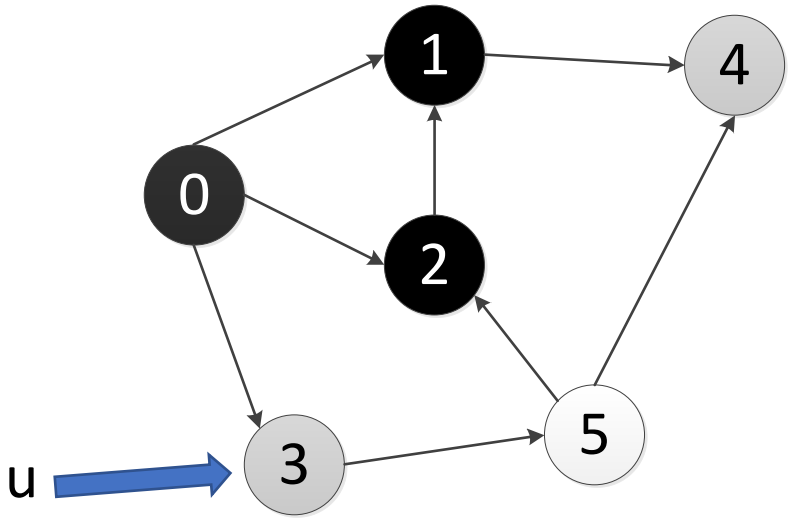
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = | 3 | 4 |

Breadth First Algorithm

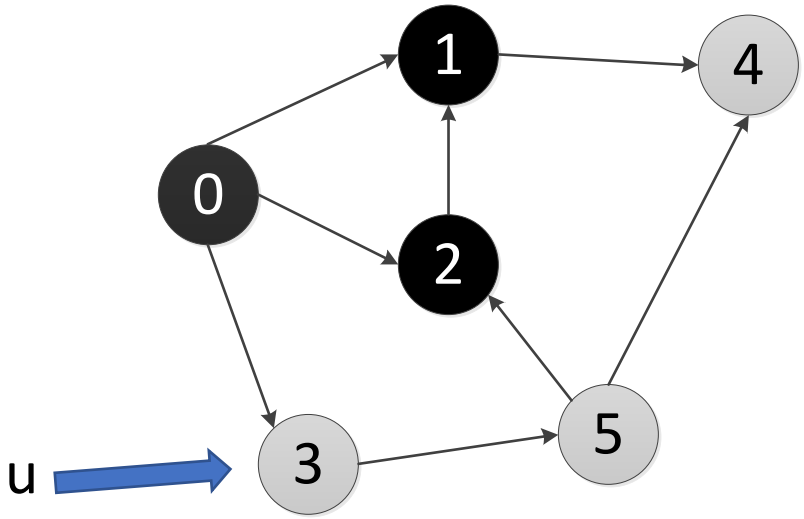
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = | 3 | 4 |

Breadth First Algorithm

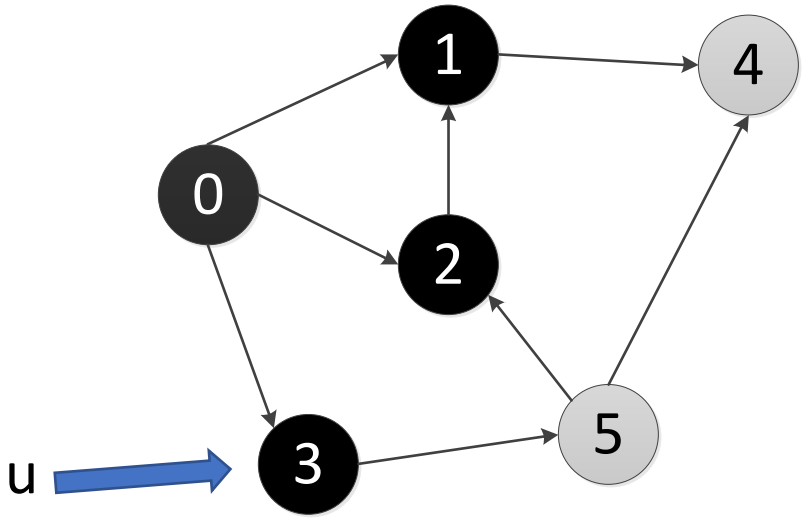
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = | 3 | 4 | 5 |

Breadth First Algorithm

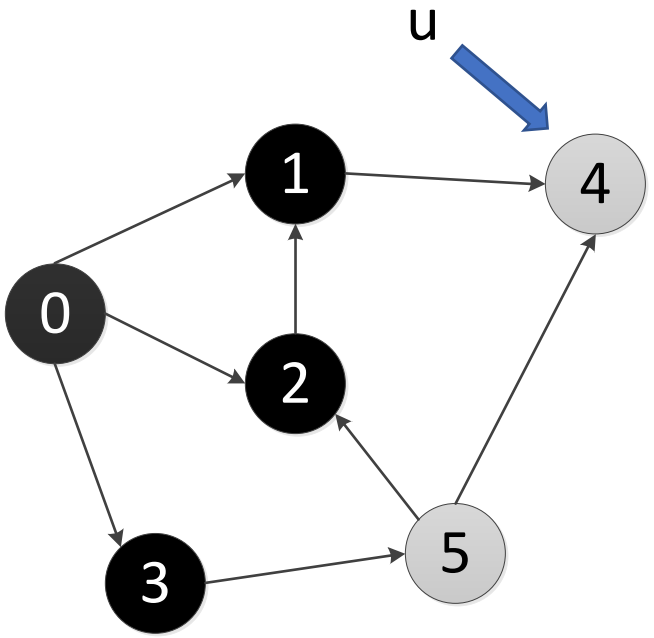
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = | 4 | 5 |

Breadth First Algorithm

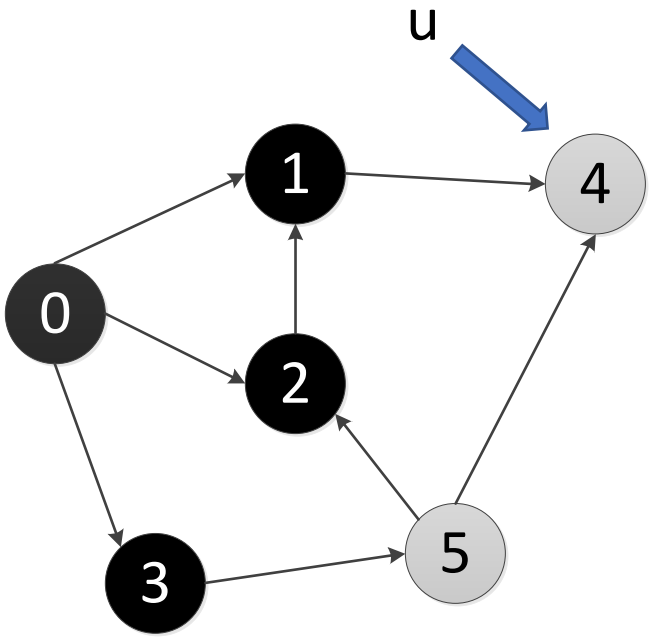
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = | 4 | 5 |

Breadth First Algorithm

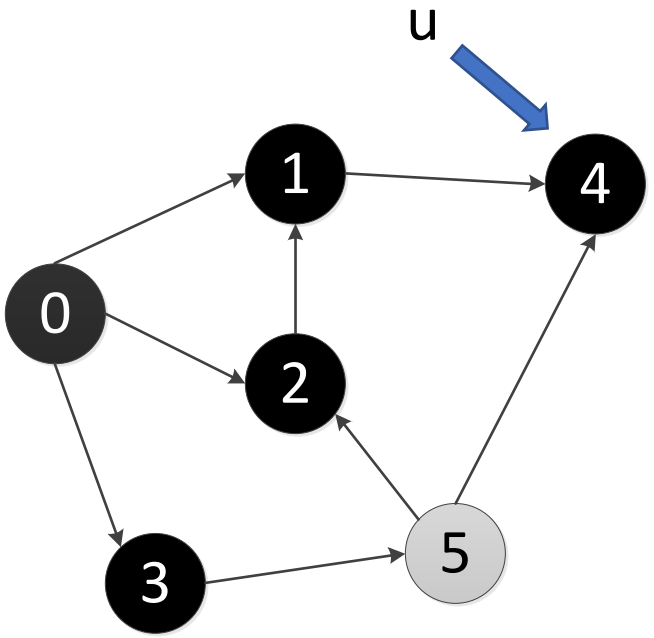
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = | 4 | 5 |

Breadth First Algorithm

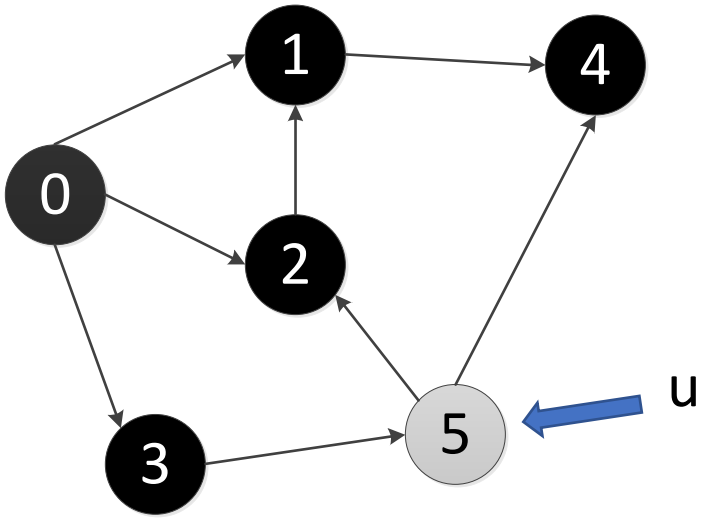
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = 5

Breadth First Algorithm

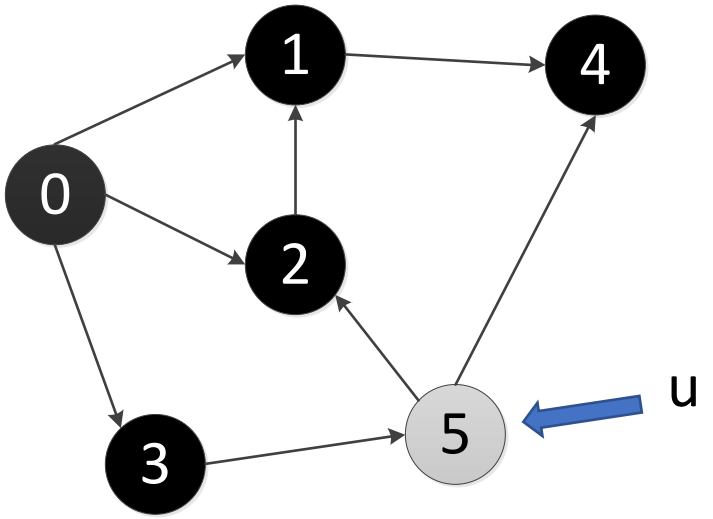
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = 5

Breadth First Algorithm

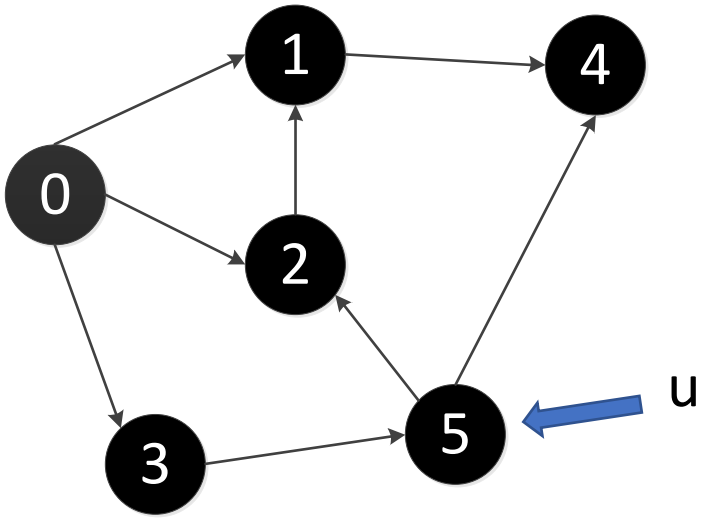
```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = 5

Breadth First Algorithm

```
Given graph G=(V,E) and source vertex s∈ V
Create a queue Q
For each vertex u ∈ V – {s}
    color[u] ← white
color[s] ← gray
Q←{s}
While Q ≠ ∅
{
    u ← head[Q];
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Enqueue(Q,v)
        }
    Dequeue(Q)
    color[u] ← black;
}
```



Q = ∅

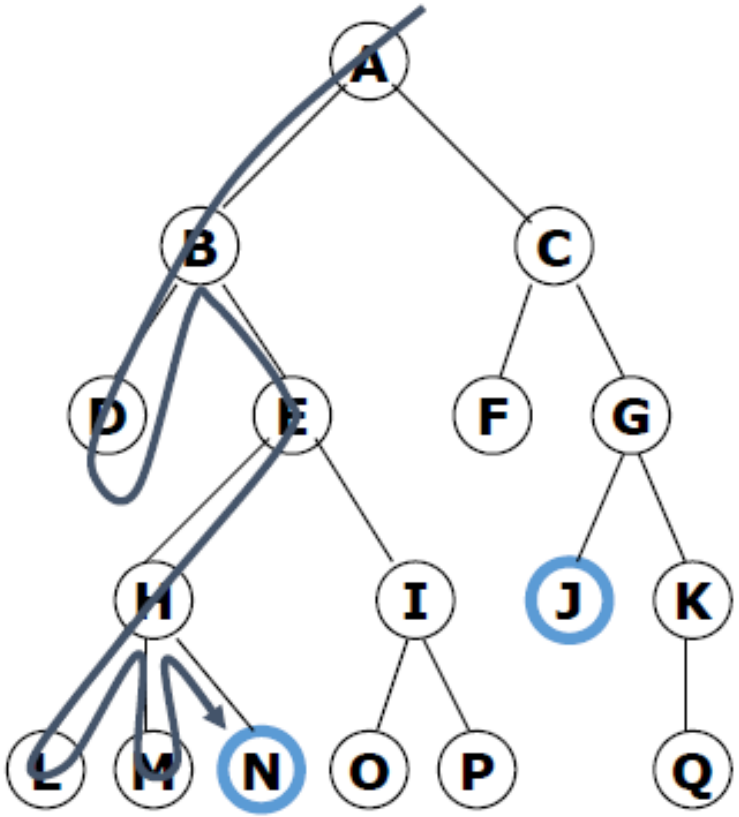
Some Applications of BFS

- Find a shortest path in terms of hops from a vertex **s** to a vertex **v**
- Find the length of such a path
- Web crawling up-to some level

Depth First Search

- Depth-first search:
  - Strategy Go as deep as can, visiting un-visited nodes
- Choose any un-visited vertex when you have a choice
- When stuck at a dead-end, backtrack as little as possible
- Back up to where you could go to another unvisited vertex
- Then continue to go on from that point
- Eventually you'll return to where you started

Depth First Search



- A depth-first search (DFS) explores a path all the way to a leaf before backtracking and exploring another path
- For example, after searching A, then B, then D, the search backtracks and tries another path from B
- Node are explored in the order A B D E H L M N I O P C F G J K Q
- N will be found before J

DFS Iterative Algorithm

```
Given graph G = (V, E) and source vertex s ∈ V
Create a Stack S
for each vertex u ∈ V – { s }
    color[u] ← white
color[s] ← gray
S ← { s }
While S ≠ ∅
{
    u ← top[S]
    print Pop(S)
    for each v ∈ Adjacent[u]
        if color[v] = white
        {
            color[v] ← gray
            Push(S,v)
        }
    color[u] ← black
}
```

Applications of DFS

- Topological sorting (in DAGs)
- Finding strongly connected components
- Determine whether two nodes, node *x* and node *y*, have a path between them

Time complexity

- The Time complexity of both BFS and DFS will be **O(V + E)**

Task

